

```

import re

import json

import math

import sqlite3

import os

from datetime import datetime

from flask import Flask, request, jsonify, render_template, send_from_directory

from werkzeug.utils import secure_filename

app = Flask(__name__)

app.config['UPLOAD_FOLDER'] = 'uploads'

app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024 # 5 MB max

ALLOWED_EXTENSIONS = {'txt', 'pdf'}


os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)


# _____

# Database

# _____

DB_PATH = 'scam_results.db'

def init_db():

    conn = sqlite3.connect(DB_PATH)

    c = conn.cursor()

    c.execute("""

        CREATE TABLE IF NOT EXISTS results (

            id      INTEGER PRIMARY KEY AUTOINCREMENT,

            snippet  TEXT,

            risk_level TEXT,

            score    REAL,

            explanation TEXT,

```

```

        flags    TEXT,

        created_at TEXT
    )
    """)
conn.commit()

conn.close()

init_db()

def save_result(snippet, risk_level, score, explanation, flags):

    conn = sqlite3.connect(DB_PATH)

    c = conn.cursor()

    c.execute("""

        INSERT INTO results (snippet, risk_level, score, explanation, flags, created_at)

        VALUES (?, ?, ?, ?, ?, ?)

    """, (

        snippet[:300],

        risk_level,

        round(score, 2),

        explanation,

        json.dumps(flags),

        datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    ))

    conn.commit()

    conn.close()


def get_history(limit=20):

    conn = sqlite3.connect(DB_PATH)

    c = conn.cursor()

    c.execute('SELECT id, snippet, risk_level, score, explanation, flags, created_at FROM results ORDER BY id DESC

LIMIT ?', (limit,))

    rows = c.fetchall()

```

```

conn.close()

return [
    {
        'id': r[0],
        'snippet': r[1],
        'risk_level': r[2],
        'score': r[3],
        'explanation': r[4],
        'flags': json.loads(r[5]),
        'created_at': r[6]
    }
    for r in rows
]

```

```

# -----
# Risk Analysis Engine
# -----

```

```

HIGH_RISK_PATTERNS = [
    (r"\bpay\s+(registration|joining|training|security)\s+fee\b", 8, "Requests upfront payment/registration fee"),
    (r"\b(registration|joining|security|processing)\s+fee\b", 7, "Mentions suspicious fee"),
    (r"\bno\s+interview\b", 7, "Claims no interview required"),
    (r"\bwork\s+from\s+home\b.{0,60}\b(high|huge|massive|unlimited)\s+salary\b", 8, "Unrealistic work-from-home salary claim"),
    (r"\b(100%|guaranteed)\s+(job|placement|income|salary)\b", 7, "Guarantees job/income (unrealistic)"),
    (r"\bdeposit\s+(money|funds|amount)\b", 8, "Asks to deposit money"),
    (r"\bsend\s+(cash|money|payment|wire|transfer)\b", 9, "Requests direct money transfer"),
    (r"\bgift\s+card(s)?\b", 9, "Mentions gift cards as payment"),
    (r"\bcash\s+advance\b", 7, "Offers suspicious cash advance"),
    (r"\bwork\s+from\s+home\b.{0,40}\bno\s+experience\b", 7, "Work from home with no experience required"),

```

(r'\bmake\s+\\$[\d,]+\s+(per\s+day|daily|a\s+day)\b', 8, "Unrealistic daily earnings claim"),

(r'\b(earn|make)\s+\\$[\d,]{5,}\s+(per|a)\s+(week|month|day)\b', 8, "Extremely high earning promises"),

(r'\blimited\s+time\s+offer\b', 5, "Pressure tactic: limited time offer"),

(r'\bact\s+now\b', 4, "Pressure tactic: act now"),

(r'\burgent\s+(hiring|requirement|opening|vacancy)\b', 6, "Urgency pressure tactic"),

(r'\bimmediately\s+join\b', 5, "Immediate joining pressure"),

(r'\bwhatsapp\s+(me|us|only)\b', 5, "Requests WhatsApp-only contact"),

(r'\bgmail\.com|yahoo\.com|hotmail\.com|outlook\.com', 4, "Uses free email domain (not corporate)"),

(r'\bno\s+experience\s+(needed|required|necessary)\b', 5, "No experience required (suspicious context)"),

(r'\bpart.?time\b.{0,50}\b\$[\d,]{4,}\b', 6, "Part-time job with very high salary"),

(r'\b(mlm|multi.?level\s+marketing|pyramid)\b', 9, "MLM / pyramid scheme indicator"),

(r'\bpassive\s+income\b', 5, "Passive income promise"),

(

r'\b(investment|invest)\s+\\$[\d,]+\b', 7, "Requires financial investment"),

(r'\bclick\s+here\s+to\s+(apply|earn|start)\b', 4, "Suspicious call-to-action link"),

(r'\bverify\s+(your\s+)?(bank|account|details)\b', 8, "Requests bank/account verification"),

(r'\bsocial\s+security\s+(number|no\.\?)\b', 9, "Requests SSN — identity theft risk"),

(r'\bpassport\s+(copy|scan|number)\b', 7, "Requests passport details prematurely"),

(r'\bOTP\b', 6, "Mentions OTP (possible phishing)"),

]

MEDIUM_RISK_PATTERNS = [

(r'\bwork\s+from\s+home\b', 3, "Work from home (verify legitimacy)"),

(r'\bflexible\s+(hours|schedule|timing)\b', 2, "Flexible hours claim (common in scams)"),

(r'\bno\s+degree\s+(required|needed)\b', 2, "No degree required"),

(r'\bfree\s+training\b', 2, "Offers free training"),

(r'\beasy\s+(money|job|work|income)\b', 4, "Promises easy money/job"),

(r'\b(double|triple)\s+your\s+(income|salary|earnings)\b', 6, "Claims to double/triple income"),

(r'\bwork\s+at\s+your\s+own\s+pace\b', 2, "Vague work structure"),

```
(r'\bcontact\s+us\s+on\s+whatsapp\b', 3, "WhatsApp-only contact"),
(r'\bno\s+target\b', 2, "No target/performance metrics"),
(r'\bdata\s+entry\b.{0,60}\bhigh\s+(salary|pay|income)\b', 5, "High pay for data entry (often scam)",
]
```

```
LOW_RISK_PATTERNS = [
```

```
(r'\bapply\s+(online|now)\b', 1, "Standard job application phrase"),
(r'\b(competitive|attractive)\s+(salary|package|compensation)\b', 1, "Standard salary description"),
(r'\bgood\s+communication\s+skills\b', 0, "Common legitimate job requirement"),
(r'\bequal\s+opportunity\s+employer\b', -2, "Positive: Equal opportunity employer"),
(r'\bcompany\s+(website|portal|career\s+page)\b', -2, "References official company website"),
(r'\b(linkedin|naukri|indeed|glassdoor)\b', -2, "Mentions reputable job platform"),
(r'\binterview\s+(process|round|schedule)\b', -3, "Mentions legitimate interview process"),
(r'\bHR\s+(team|department|manager)\b', -1, "References HR department"),
(r'\bprovident\s+fund\b|\bPF\b|\bESI\b|\bgratuity\b', -2, "Mentions statutory benefits"),
(r'\bbackground\s+(check|verification)\b', -2, "Background check (legitimate signal)",
]
```

```
SALARY_REGEX = re.compile(
    r'(?:(?:USD|INR|₹|\$|Rs\.)?)s*([d,]+)s*(?:per|a|/)\s*(month|year| annum|day|week)',
    re.IGNORECASE
)
```

```
def check_salary_realism(text):
    """Detect suspiciously high salary mentions."""
    findings = []
    for match in SALARY_REGEX.finditer(text):
        raw = match.group(1).replace(',', '')
        try:
```

```

amount = int(raw)

except ValueError:

    continue

period = match.group(2).lower()

annual = amount

if period in ('month',):

    annual = amount * 12

elif period in ('week',):

    annual = amount * 52

elif period in ('day',):

    annual = amount * 250

if annual > 500000: # > $500k / year

    findings.append((10, f"Extremely unrealistic salary: {match.group(0).strip()}"))

elif annual > 200000:

    findings.append((5, f"Unusually high salary: {match.group(0).strip()}"))

return findings

def check_fake_domain(text):

    """Flag free email domains used as company contact."""

    findings = []

    email_matches = re.findall(r'[\w.+~]+@([\w.-]+)', text)

    free_domains = {'gmail.com', 'yahoo.com', 'hotmail.com', 'outlook.com', 'rediffmail.com', 'ymail.com'}

    for domain in email_matches:

        if domain.lower() in free_domains:

            findings.append((5, f"Company contact uses free email domain: @{domain}"))

    return findings

def analyze_text(text: str):

    text_lower = text.lower()

    score = 0.0

    flags = []

```

```

matched_explanations = []

# High-risk patterns
for pattern, weight, explanation in HIGH_RISK_PATTERNS:
    if re.search(pattern, text_lower):
        score += weight
        flags.append({'type': 'high', 'text': explanation})
        matched_explanations.append(explanation)

# Medium-risk patterns
for pattern, weight, explanation in MEDIUM_RISK_PATTERNS:
    if re.search(pattern, text_lower):
        score += weight
        flags.append({'type': 'medium', 'text': explanation})
        matched_explanations.append(explanation)

# Low-risk / positive signals
for pattern, weight, explanation in LOW_RISK_PATTERNS:
    if re.search(pattern, text_lower):
        score += weight # weight can be negative (positive signal)
        flags.append({'type': 'low', 'text': explanation})

# Salary check
for w, exp in check_salary_realism(text):
    score += w
    flags.append({'type': 'high', 'text': exp})
    matched_explanations.append(exp)

# Email domain check
for w, exp in check_fake_domain(text):

```

```
    score += w

    flags.append({'type': 'medium', 'text': exp})

    matched_explanations.append(exp)


# Clamp score
score = max(0, score)


# Normalize score to 0-100%
max_possible = 60
probability = min(100, round((score / max_possible) * 100, 1))


# Risk level classification
if probability >= 55:
    risk_level = 'HIGH'
    color = 'red'
    icon = ""
elif probability >= 25:
    risk_level = 'MEDIUM'
    color = 'yellow'
    icon = ""
else:
    risk_level = 'LOW'
    color = 'green'
    icon = ""


# Build human-readable explanation
if not matched_explanations:
    if risk_level == 'LOW':
```



```
explanation = "No significant scam indicators were found. The message appears to be from a legitimate source."
```

```
else:
```

```
    explanation = "Some patterns match known scam characteristics. Proceed with caution."
```

```
else:
```

```
    top = matched_explanations[:4]
```

```
    explanation = "Detected: " + "; ".join(top) + "."
```

```
    if len(matched_explanations) > 4:
```

```
        explanation += f" ({len(matched_explanations)-4} more indicators)"
```

```
return {
```

```
    'risk_level': risk_level,
```

```
    'probability': probability,
```

```
    'color': color,
```

```
    'icon': icon,
```

```
    'explanation': explanation,
```

```
    'flags': flags,
```

```
    'raw_score': round(score, 2)
```

```
}
```

```
# _____
```

```
# Routes
```

```
# _____
```

```
def allowed_file(filename):
```

```
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

```
@app.route('/')
```

```
def index():
    return render_template('index.html')

@app.route('/api/analyze', methods=['POST'])
def analyze():
    data = request.get_json(silent=True)

    if not data or 'text' not in data:
        return jsonify({'error': 'No text provided'}), 400

    text = data['text'].strip()

    if len(text) < 10:
        return jsonify({'error': 'Text too short. Please paste a complete job message.'}), 400

    result = analyze_text(text)

    save_result(text, result['risk_level'], result['probability'], result['explanation'], result['flags'])

    return jsonify(result)

@app.route('/api/upload', methods=['POST'])
def upload():
    if 'file' not in request.files:
        return jsonify({'error': 'No file uploaded'}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({'error': 'Empty filename'}), 400

    if not allowed_file(file.filename):
```

```
return jsonify({'error': 'Only .txt and .pdf files are allowed'}), 400
```

```
filename = secure_filename(file.filename)
```

```
filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
```

```
file.save(filepath)
```

```
# Extract text
```

```
text = ""
```

```
if filename.endswith('.txt'):
```

```
    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
```

```
        text = f.read()
```

```
elif filename.endswith('.pdf'):
```

```
    try:
```

```
        import PyPDF2
```

```
        with open(filepath, 'rb') as f:
```

```
            reader = PyPDF2.PdfReader(f)
```

```
            for page in reader.pages:
```

```
                text += page.extract_text() or "
```

```
except ImportError:
```

```
    return jsonify({'error': 'PyPDF2 not installed. Install with: pip install PyPDF2'}), 500
```

```
os.remove(filepath) # clean up
```

```
if not text.strip():
```

```
    return jsonify({'error': 'Could not extract text from file'}), 400
```

```
result = analyze_text(text)
```

```
result['extracted_text'] = text[:500]

save_result(text, result['risk_level'], result['probability'], result['explanation'], result['flags'])

return jsonify(result)
```

```
@app.route('/api/history', methods=['GET'])
```

```
def history():
```

```
    return jsonify(get_history())
```

```
@app.route('/api/history/<int:record_id>', methods=['DELETE'])
```

```
def delete_record(record_id):
```

```
    conn = sqlite3.connect(DB_PATH)
```

```
    c = conn.cursor()
```

```
    c.execute('DELETE FROM results WHERE id = ?', (record_id,))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    return jsonify({'success': True})
```

```
@app.route('/api/stats', methods=['GET'])
```

```
def stats():
```

```
    conn = sqlite3.connect(DB_PATH)
```

```
    c = conn.cursor()
```

```
    c.execute('SELECT risk_level, COUNT(*) FROM results GROUP BY risk_level')
```

```
    counts = dict(c.fetchall())
```

```
    c.execute('SELECT AVG(score) FROM results')
```

```
    avg = c.fetchone()[0] or 0
```

```
    conn.close()
```

```
    return jsonify({
```

```
'total': sum(counts.values()),  
    'high': counts.get('HIGH', 0),  
    'medium': counts.get('MEDIUM', 0),  
    'low': counts.get('LOW', 0),  
    'avg_score': round(avg, 1)  
})
```

```
if __name__ == '__main__':  
    app.run(debug=True, port=5000)
```